Actividad 5. Design thinking, definiendo e ideando

Materia:	*Gestión de Proyectos de Inteligencia Artificial*
Alumno(a):	*Diego Erando Alejandre Gutiérrez*
Matrícula:	*AL07183375*
Docente:	*Luis Ariel Vázquez*
Fecha de entrega:	*[18 / 10/ 2026]*

Declaración de Uso de Inteligencia Artificial

El alumno debe indicar de manera honesta si utilizó herramientas de Inteligencia Artificial (IA) para el desarrollo de esta actividad. Completa la siguiente tabla:

Herramienta de IA utilizada	¿La usaste?	Propósito / Descripción de uso
ChatGPT / OpenAI	No	-
Claude (Anthropic)	No	-
Gemini (Google)	Sí	Generación de ideas para la actividad, búsqueda de datos y referencias para mi proyecto. Ayuda con la instalación y troubleshooting para hace funcionar MLFlow.
Copilot (Microsoft)	No	-
Otra (especifica):	No	-

Desarrollo de Contenido

1. Introducción

Se seleccionó el caso de predicción de “churning” de clientes para el tipo de modelos que se requieren para la actividad. Continuar con el tema de análisis de sentimientos no es viable para este enfoque.El dataset de **Telco Customer Churn** de IBM predice si un cliente cancelará su servicio de telecomunicaciones basándose en su comportamiento, los servicios contratados y su antigüedad.

2. Desarrollo

2.1 Descripción del dataset

El dataset Telco Customer Churn (abandono de clientes) nos sirve para poder predecir si un cliente abandonará el servicio (Si o No), apoyándose en el comportamiento histórico del cliente.

Lo obtuvimos de Kaggle (https://www.kaggle.com/datasets/blastchar/telco-customer-churn) y se apoya en los parámetros principales siguientes:

Dato	Descripción
Demográfico	De acuerdo a la edad
Servicios contratados	Número de líneas, internet, soporte, streaming.
Tipo de cuenta del cliente	Tipo de contrato, método de pago, facturación electrónica, antigüedad.
Financiero	Cargo mensual, acumulados.

2.2 Proceso de limpieza y estandarización

Se aplicaron las siguientes transformaciones o ajustes para los siguientes casos_

a) **Manejo de nulos y tipos de datos:** Se detectó que la variable `TotalCharges` venía como texto (string) con espacios en blanco para clientes nuevos. Se forzó su conversión a formato numérico y se imputaron/eliminaron esos valores nulos.

b) **Codificación de variables categóricas:** Se transformó la variable objetivo `Churn` a formato binario (1 = Yes, 0 = No). Se utilizó *One-Hot Encoding* (variables dummy) para las características nominales (como tipo de contrato, métodos de pago y servicios).
c) **Escalado:** Las variables numéricas continuas (`tenure`, `MonthlyCharges`, `TotalCharges`) fueron estandarizadas/normalizadas para evitar que el algoritmo diera mayor peso a variables con escalas más grandes.

2.3 Estrategia de versionamiento

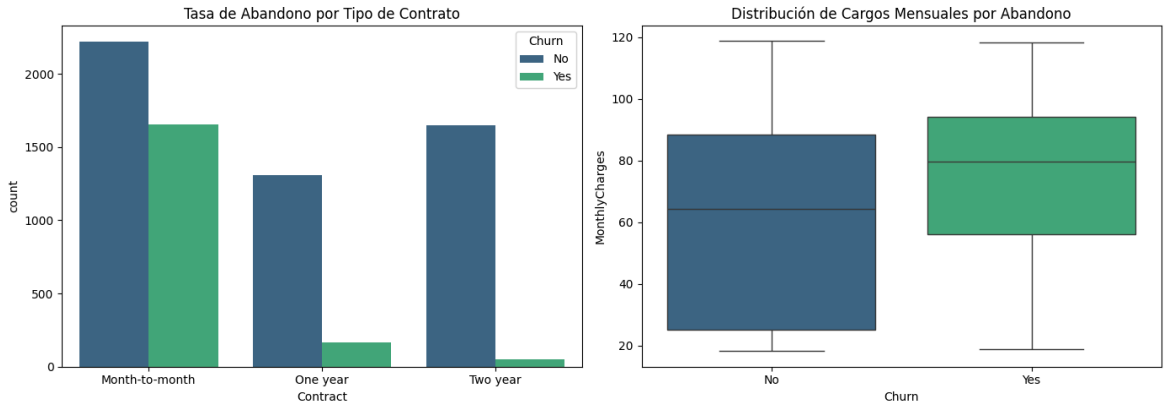
Se creó un nuevo repositorio para la actividad (Actividad_5_DEAG), con la estructura siguiente para el control de versiones:

```
Actividad5/
├── datos/
│   ├── datos_ini # datos originales
│   └── datos_limp # datos limpios
├── fuentes/
│   ├── entrena.ipynb # notebook de Colab (entrenamiento y
evaluación)
│   ├── datos_prep.py # funciones de limpieza
│   └── train.py # script de entrenamiento y registro en MLflow
├── README.md # guía de reproducción y decisiones
├── CHANGELOG.md # reporte final
```

Los cambios a nivel de código y estructura se documentaron y versionaron mediante *commits*, asegurando la reproducibilidad del flujo de trabajo desde la ingesta hasta el modelado.

2.4 Visualizaciones relevantes

Se observó que los clientes con contratos de mes a mes y aquellos con cargos mensuales más altos tienen una mayor tasa de abandono.



La primera gráfica demuestra visualmente que la inmensa mayoría de las cancelaciones de servicio (barras verdes que indican "Yes") se concentran en la categoría de contratos "Month-to-month", reduciéndose de forma drástica en los contratos de uno y dos años. La segunda gráfica revela mediante el diagrama de caja que la mediana del cargo mensual para los clientes que abandonan la compañía se sitúa cerca de 80, un valor significativamente mayor que la mediana de aproximadamente 64 correspondiente a los clientes que deciden quedarse ("No").

2.5 Descripción de los modelos entrenados

Se entrenaron y compararon dos algoritmos supervisados de clasificación:

1. Regresión Logística: Un modelo lineal, rápido y altamente interpretable, excelente para establecer una línea base de rendimiento.
2. Random Forest (Bosque Aleatorio): Un modelo de ensamble basado en múltiples árboles de decisión, capaz de capturar relaciones no lineales complejas entre las variables.

2.6 Configuración de hiperparámetros

Se utilizó GridSearchCV con validación cruzada (K-Fold = 5) para encontrar la mejor configuración. Los parámetros ganadores registrados por MLflow fueron:

Regresión Logística: C = 10, solver = 'lbfgs'.

Random Forest: max_depth = 5, n_estimators = 100.

GridSearchCV me ayudó a encontrar las mejores ajustes para los hiperparámetros, llegando a los valores de k-fold=5, los valores de regresión lógica y los valores para el random forest, sin necesidad de yo estar adivinando o ajustando manualmente.

2.7 Métricas obtenidas

Los resultados extraídos directamente de MLflow son los siguientes:

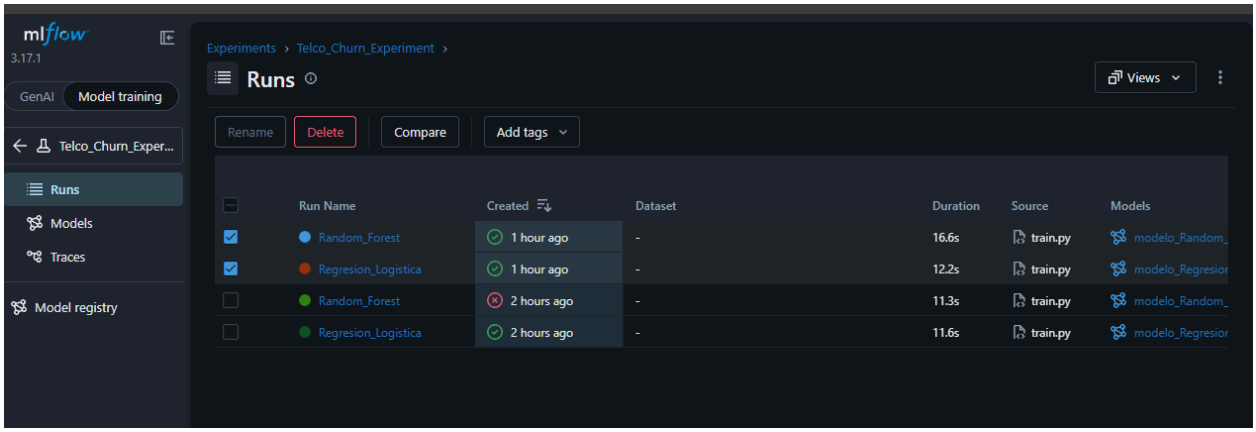
- **Regresión Logística:**
  - Accuracy: 0.820
  - Precision: 0.683
  - Recall: **0.595**
  - F1-Score: 0.636
  - ROC-AUC: 0.862
- **Random Forest:**
  - Accuracy: 0.801
  - Precision: 0.702
  - Recall: 0.429
  - F1-Score: 0.532
  - ROC-AUC: 0.864

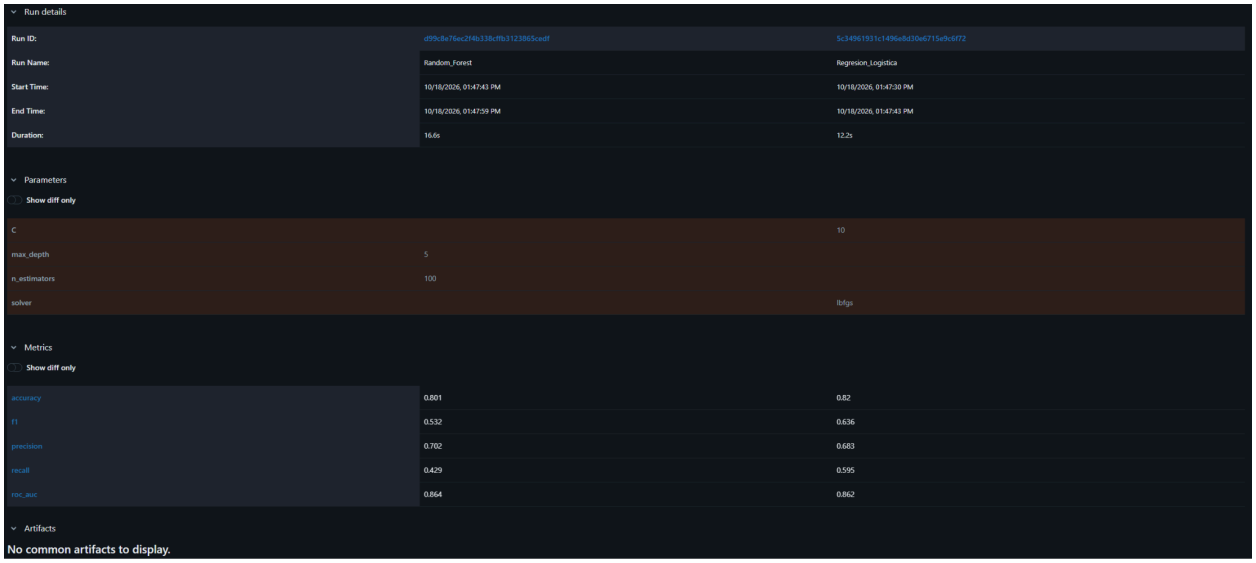
2.7 Comparación estadística de resultados

De acuerdo a los resultados, la Regresión Logística es el modelo más adecuado. En un problema de negocio enfocado en el abandono de clientes (Churn), el error financiero más grave es no detectar a un cliente que está a punto de irse (falso negativo). Por lo tanto, la métrica crítica a optimizar es el Recall. La Regresión Logística alcanzó un Recall de 0.595, superando por mucho el deficiente 0.429 del Random Forest. Aunque el Random Forest fue ligeramente superior en ROC-AUC y Precisión, su incapacidad para identificar correctamente los casos positivos reales de fuga lo descarta para este escenario. Además, la Regresión Logística logró un mejor equilibrio global con un F1-Score de 0.636 frente al 0.532 del modelo de ensamble.

2.8 Evidencia del uso de MLflow

En esta parte me apoyé mucho de Gemini para poder ejecutar de manera remota la herramienta. Al final se pudo obtener lo necesario:





2.9 Reflexión sobre deuda técnica y MLOps

La deuda técnica en Machine Learning ocurre cuando se prioriza la creación rápida de modelos sobre la infraestructura para mantenerlos, auditarlos y desplegarlos. El rastreo manual de experimentos (como anotar hiperparámetros en Excel) provoca que los modelos sean irreproducibles, aumentando el riesgo operativo en producción. La implementación de herramientas MLOps como **MLflow** en esta actividad mitiga esto mediante:

- **Trazabilidad Inmutable:** Registra automáticamente el historial completo de ejecuciones en la base de datos (SQLite), lo que permite auditar y justificar por qué se eligió un modelo sobre otro con evidencia matemática exacta.
- **Reproducibilidad:** Empaqueta las versiones del código, los parámetros de `GridSearchCV` y las dependencias, garantizando que el modelo se comporte igual en desarrollo que en producción.

- **Gestión Centralizada:** Almacena métricas y artefactos (como la matriz de confusión y el modelo serializado) en un solo lugar, eliminando la fricción de entrega entre los científicos de datos y los ingenieros de software.